

radioactiveSim

Generated by Doxygen 1.16.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Box Class Reference	5
3.2 lazy_node< T > Struct Template Reference	5
3.2.1 Detailed Description	6
3.2.2 Member Function Documentation	6
3.2.2.1 BH_criteria() [1/2]	6
3.2.2.2 BH_criteria() [2/2]	7
3.2.2.3 Calc_BH_criteria()	7
3.2.2.4 child_index()	7
3.2.2.5 eval()	8
3.2.2.6 get()	8
3.2.2.7 instantiate_child()	8
3.2.2.8 read()	8
3.3 Material Struct Reference	9
3.3.1 Detailed Description	9
3.3.2 Constructor & Destructor Documentation	9
3.3.2.1 Material()	9
3.4 Octree Class Reference	10
3.4.1 Detailed Description	10
3.4.2 Constructor & Destructor Documentation	10
3.4.2.1 Octree()	10
3.4.3 Member Function Documentation	11
3.4.3.1 add_particle()	11
3.4.3.2 build()	11
3.4.3.3 calc_force()	11
3.4.3.4 get_root()	12
3.4.3.5 set_default_field()	12
3.4.3.6 set_theta()	13
3.5 Particle Class Reference	13
3.5.1 Detailed Description	14
3.5.2 Constructor & Destructor Documentation	14
3.5.2.1 Particle() [1/3]	14
3.5.2.2 Particle() [2/3]	14
3.5.2.3 Particle() [3/3]	15
3.5.3 Member Function Documentation	16
3.5.3.1 move()	16
3.6 ROI Class Reference	17

3.6.1 Detailed Description	17
3.6.2 Constructor & Destructor Documentation	18
3.6.2.1 ROI()	18
3.6.3 Member Function Documentation	19
3.6.3.1 contains()	19
3.6.3.2 score()	19
3.7 Shielding Class Reference	20
3.7.1 Detailed Description	20
3.7.2 Constructor & Destructor Documentation	20
3.7.2.1 Shielding()	20
3.7.3 Member Function Documentation	21
3.7.3.1 contains()	21
3.8 Simulation Class Reference	22
3.8.1 Detailed Description	23
3.8.2 Constructor & Destructor Documentation	23
3.8.2.1 Simulation()	23
3.8.3 Member Function Documentation	23
3.8.3.1 print()	23
3.8.3.2 run()	24
3.9 Source Class Reference	24
3.9.1 Detailed Description	25
3.9.2 Constructor & Destructor Documentation	25
3.9.2.1 Source()	25
3.9.3 Member Function Documentation	25
3.9.3.1 emit()	25
3.10 vector3< T > Struct Template Reference	26
3.10.1 Detailed Description	27
3.10.2 Member Function Documentation	27
3.10.2.1 norm()	27
3.10.2.2 operator*() [1/2]	27
3.10.2.3 operator*() [2/2]	28
3.10.2.4 operator*=() [1/2]	28
3.10.2.5 operator*=() [2/2]	28
3.10.2.6 operator+()	29
3.10.2.7 operator+=()	29
3.10.2.8 operator-()	29
3.10.2.9 operator-=()	30
3.10.2.10 operator/()	30
3.10.2.11 operator/=()	30
3.10.2.12 operator==()	31
3.10.3 Friends And Related Symbol Documentation	31
3.10.3.1 operator*	31

4 File Documentation	33
4.1 Box.h	33
4.2 src/GeometryOutput.cpp File Reference	34
4.2.1 Detailed Description	34
4.2.2 Function Documentation	34
4.2.2.1 is_in_bounding_box()	34
4.2.2.2 write_box_vtp()	35
4.3 src/GeometryOutput.h File Reference	35
4.3.1 Detailed Description	35
4.3.2 Function Documentation	36
4.3.2.1 is_in_bounding_box()	36
4.3.2.2 write_box_vtp()	36
4.4 GeometryOutput.h	37
4.5 src/main.cpp File Reference	37
4.5.1 Detailed Description	37
4.6 src/Material.h File Reference	37
4.6.1 Detailed Description	38
4.7 Material.h	38
4.8 src/Octree.cpp File Reference	38
4.8.1 Detailed Description	38
4.9 src/Octree.h File Reference	38
4.9.1 Detailed Description	39
4.10 Octree.h	39
4.11 src/Output.cpp File Reference	41
4.11.1 Detailed Description	41
4.11.2 Function Documentation	42
4.11.2.1 write_particles_csv()	42
4.12 src/Output.h File Reference	42
4.12.1 Detailed Description	42
4.12.2 Function Documentation	42
4.12.2.1 write_particles_csv()	42
4.13 Output.h	43
4.14 src/Particle.cpp File Reference	43
4.14.1 Detailed Description	43
4.15 src/Particle.h File Reference	43
4.15.1 Detailed Description	44
4.16 Particle.h	44
4.17 src/ROI.cpp File Reference	44
4.17.1 Detailed Description	44
4.18 src/ROI.h File Reference	45
4.18.1 Detailed Description	45
4.19 ROI.h	45

4.20 src/Shielding.cpp File Reference	45
4.20.1 Detailed Description	45
4.21 src/Shielding.h File Reference	46
4.21.1 Detailed Description	46
4.22 Shielding.h	46
4.23 src/Simulation.cpp File Reference	46
4.23.1 Detailed Description	47
4.24 src/Simulation.h File Reference	47
4.24.1 Detailed Description	47
4.25 Simulation.h	47
4.26 src/Source.cpp File Reference	48
4.26.1 Detailed Description	48
4.27 src/Source.h File Reference	48
4.27.1 Detailed Description	48
4.28 Source.h	48
4.29 src/test.cpp File Reference	49
4.29.1 Detailed Description	49
4.30 src/utils.h File Reference	49
4.30.1 Detailed Description	50
4.30.2 Function Documentation	50
4.30.2.1 distance()	50
4.31 utils.h	51
Index	53

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Box	5
lazy_node< T >	
A lazy node for the octree data structure	5
Material	
Describes the physical properties of a shielding material	9
Octree	
A spatial partitioning octree data structure for implementing the Barnes-Hut algorithm	10
Particle	
Represents a single physical particle moving through the simulation domain	13
ROI	
An axis-aligned bounding box used to score particles entering the region	17
Shielding	
An axis-aligned rectangular volume of shielding material	20
Simulation	
Top-level simulation driver	22
Source	
Models a radioactive point source that emits particles every time step	24
vector3< T >	
A generic 3-D vector whose component type is T	26

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/ Box.h	33
src/ GeometryOutput.cpp	
Implementation of geometry output utilities: VTP mesh writing and global bounding-box particle culling	34
src/ GeometryOutput.h	
Declarations for geometry output utilities: writing bounding-box meshes in VTP format and enforcing the simulation domain boundary	35
src/ main.cpp	
Entry point for the radioactive particle transport simulation	37
src/ Material.h	
Declaration of the Material struct, which describes the physical properties of a shielding material relevant to particle attenuation	37
src/ Octree.cpp	
This file contains the implementation of the Octree class and its member functions	38
src/ Octree.h	
This file contains the definition of the Octree class and lazy_node	38
src/ Output.cpp	
Implementation of CSV output utilities for particle state persistence	41
src/ Output.h	
Declarations for CSV output utilities used to persist particle state	42
src/ Particle.cpp	
Implementation of the Particle class methods	43
src/ Particle.h	
Declaration of the Particle class, representing a physical particle (e.g. proton or neutron) tracked throughout the simulation	43
src/ ROI.cpp	
Implementation of the ROI (Region Of Interest) class methods	44
src/ ROI.h	
Declaration of the ROI (Region Of Interest) class, which defines an axis-aligned box used to score particles that pass through it	45
src/ Shielding.cpp	
Implementation of the Shielding class methods	45
src/ Shielding.h	
Declaration of the Shielding class, which models an axis-aligned rectangular shielding volume filled with a given material	46

src/ Simulation.cpp	Implementation of the Simulation class methods	46
src/ Simulation.h	Declaration of the Simulation class, which orchestrates the full radioactive particle transport simulation	47
src/ Source.cpp	Implementation of the Source class methods	48
src/ Source.h	Declaration of the Source class, which models a radioactive point source that emits particles isotropically at a configurable rate	48
src/ test.cpp	This file contains unit tests for the utility functions and data structures used in the simulation . .	49
src/ utils.h	Utility types and functions shared across the simulation	49

Chapter 3

Class Documentation

3.1 Box Class Reference

Public Member Functions

- **Box** (double x1, double x2, double y1, double y2, double z1, double z2)
- **Box** ([vector3](#)< double > minVec, [vector3](#)< double > maxVec)
- bool **contains** (const [Particle](#) &p) const

The documentation for this class was generated from the following file:

- src/Box.h

3.2 lazy_node< T > Struct Template Reference

A lazy node for the octree data structure.

```
#include <Octree.h>
```

Public Member Functions

- constexpr size_t **child_index** (const [vector3](#)< T > &p) const
- constexpr bool **is_child_inst** (const size_t &n) const
Return True if nth child is evaluated (exist).
- constexpr bool **is_leaf** () const
Return True if the node is a leaf (has no children).
- constexpr double **Calc_BH_criteria** ([vector3](#)< T > position) const
Calculate the barnes hut criteria for the current node and a given position.
- constexpr bool **BH_criteria** (const T &x, const T &y, const T &z, float theta) const
Return True if the barnes hut criteria is satisfied current node.
- constexpr bool **BH_criteria** ([vector3](#)< T > position, float theta) const
Return True if the barnes hut criteria is satisfied current node.
- void **eval** ()

- *Calculate the center of mass and total charge.*
void [eval](#) (const size_t &n)
- *Calculate the center of mass and total charge of the n-th child.*
void [instantiate_child](#) (const size_t &n)
- *Instantiate the nth child if it is not already instantiated.*
const [lazy_node](#) & [read](#) (const size_t &n) const
- *Return the nth child. This function is read-only and will throw an error if the child is not instantiated.*
[lazy_node](#) & [get](#) (const size_t &n)
- *Return the nth child.*

Public Attributes

- struct {
 T **length**
 [vector3](#)< T > **center**
} **bounding_box**
- [vector3](#)< T > **center_of_charge**
- int **total_charge**
- std::unique_ptr< [lazy_node](#)< T > > **childs** [8]
- std::shared_ptr< [Particle](#) > **particle**

3.2.1 Detailed Description

```
template<typename T>
struct lazy_node< T >
```

A lazy node for the octree data structure.

Template Parameters

<i>T</i>	The type of the coordinates. Must be a numeric type (int, float, double, etc.).
----------	---

Note

Spatial indexing is done using morton indices, which means that the children of a node are indexed as follows: 0: lower-left-back 1: lower-right-back 2: upper-left-back 3: upper-right-back 4: lower-left-front 5: lower-right-front 6: upper-left-front 7: upper-right-front

3.2.2 Member Function Documentation

3.2.2.1 BH_criteria() [1/2]

```
template<typename T>
bool lazy_node< T >::BH_criteria (
    const T & x,
    const T & y,
    const T & z,
    float theta) const [inline], [constexpr]
```

Return True if the barnes hut criteria is satisfied current node.

Parameters

<i>x</i>	X coordinate of the position of the particle we want to calculate the force from this node on.
----------	--

<i>y</i>	Y coordinate of the position of the particle we want to calculate the force from this node on.
<i>z</i>	Z coordinate of the position of the particle we want to calculate the force from this node on.
<i>theta</i>	Theta parameter for the barnes hut criteria.

3.2.2.2 BH_criteria() [2/2]

```
template<typename T>
bool lazy_node< T >::BH_criteria (
    vector3< T > position,
    float theta) const [inline], [constexpr]
```

Return True if the barnes hut criteria is satisfied current node.

Parameters

<i>position</i>	Position of the particle we want to calculate the force from this node on.
<i>theta</i>	Theta parameter for the barnes hut criteria.

3.2.2.3 Calc_BH_criteria()

```
template<typename T>
double lazy_node< T >::Calc_BH_criteria (
    vector3< T > position) const [inline], [constexpr]
```

Calculate the barnes hut criteria for the current node and a given position.

Parameters

<i>position</i>	Position of the particle we want to calculate the force from this node on.
-----------------	--

Returns

The barnes hut criteria for the current node and the given position.

3.2.2.4 child_index()

```
template<typename T>
size_t lazy_node< T >::child_index (
    const vector3< T > & p) const [inline], [constexpr]
```

Parameters

<i>p</i>	The position of the particle.
----------	-------------------------------

Returns

The child index of the particle position using morton indices.

Exceptions

<i>std::runtime_error</i>	if the particle position is not within the bounding box of the node.
---------------------------	--

3.2.2.5 eval()

```
template<typename T>
void lazy_node< T >::eval (
    const size_t & n) [inline]
```

Calculate the center of mass and total charge of the n-th child.

Parameters

<i>n</i>	The index of the child to evaluate. The indices are defined by the morton indices.
----------	--

3.2.2.6 get()

```
template<typename T>
lazy_node & lazy_node< T >::get (
    const size_t & n) [inline]
```

Return the nth child.

Parameters

<i>n</i>	The index of the child to return. The indices are defined by the morton indices.
----------	--

3.2.2.7 instantiate_child()

```
template<typename T>
void lazy_node< T >::instantiate_child (
    const size_t & n) [inline]
```

Instantiate the nth child if it is not already instantiated.

Parameters

<i>n</i>	The index of the child to instantiate. The indices are defined by the morton indices.
----------	---

3.2.2.8 read()

```
template<typename T>
const lazy_node & lazy_node< T >::read (
    const size_t & n) const [inline]
```

Return the nth child. This function is read-only and will throw an error if the child is not instantiated.

Parameters

<i>n</i>	The index of the child to return. The indices are defined by the morton indices.
----------	--

The documentation for this struct was generated from the following file:

- [src/Octree.h](#)

3.3 Material Struct Reference

Describes the physical properties of a shielding material.

```
#include <Material.h>
```

Public Member Functions

- [Material](#) (std::string n="Vacuum", double mfp=1e9)
Construct a material with a given name and mean free path.

Public Attributes

- std::string **name**
Human-readable name of the material (e.g. "Lead", "Vacuum").
- double **mean_free_path**
Mean free path of the material in metres. Larger values mean the material is more transparent to radiation. Defaults to 1e9 (effectively transparent, i.e. vacuum).

3.3.1 Detailed Description

Describes the physical properties of a shielding material.

Currently the only property used by the simulation is the mean free path, which governs exponential attenuation of particles passing through the material.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 Material()

```
Material::Material (
    std::string n = "Vacuum",
    double mfp = 1e9) [inline]
```

Construct a material with a given name and mean free path.

Parameters

<i>n</i>	Name of the material. Defaults to "Vacuum".
<i>mfp</i>	Mean free path in metres. Defaults to 1e9 (vacuum).

The documentation for this struct was generated from the following file:

- src/[Material.h](#)

3.4 Octree Class Reference

A spatial partitioning octree data structure for implementing the Barnes-Hut algorithm.

```
#include <Octree.h>
```

Public Member Functions

- **Octree** (vector3< nt > minVec, vector3< nt > maxVec)
*Construct an **Octree** covering the axis-aligned box defined by minVec and maxVec. The root node's bounding box is the smallest cube that contains the input box.*
- void **set_default_field** (Octree def)
Replace the default field of the octree (reserved for future use).
- void **add_particle** (const Particle &p)
Add a single particle to the octree (public wrapper).
- void **calc_force** (Particle &p)
Calculate the electrostatic force acting on p using the Barnes-Hut approximation and store the result in p.↔ acceleration.
- void **build** (const std::vector< Particle > &particles)
Build the entire octree from a flat vector of particles. Equivalent to calling add_particle() for each particle in particles.
- void **set_theta** (float t)
Set the Barnes-Hut opening-angle parameter.
- float **get_theta** () const
Return the current Barnes-Hut opening-angle parameter.
- **lazy_node**< nt > &**get_root** ()
Return a mutable reference to the root node of the octree.

3.4.1 Detailed Description

A spatial partitioning octree data structure for implementing the Barnes-Hut algorithm.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 Octree()

```
Octree::Octree (
    vector3< nt > minVec,
    vector3< nt > maxVec)
```

Construct an **Octree** covering the axis-aligned box defined by minVec and maxVec. The root node's bounding box is the smallest cube that contains the input box.

Construct an **Octree** that covers the axis-aligned box [minVec, maxVec]. The root bounding box becomes the smallest cube enclosing that box.

Parameters

<i>minVec</i>	Minimum corner of the initial bounding volume.
---------------	--

<i>maxVec</i>	Maximum corner of the initial bounding volume.
<i>minVec</i>	Minimum corner of the region to partition.
<i>maxVec</i>	Maximum corner of the region to partition.

3.4.3 Member Function Documentation

3.4.3.1 add_particle()

```
void Octree::add_particle (
    const Particle & p)
```

Add a single particle to the octree (public wrapper).

Add a particle to the octree. This is a public wrapper function for the private `add_particle` function that takes a node as an argument.

Parameters

<i>p</i>	The particle to insert.
<i>p</i>	The particle to add to the octree.

3.4.3.2 build()

```
void Octree::build (
    const std::vector< Particle > & particles)
```

Build the entire octree from a flat vector of particles. Equivalent to calling `add_particle()` for each particle in `particles`.

Build the octree from a vector of particles.

Parameters

<i>particles</i>	The collection of particles to insert into the octree.
<i>particles</i>	The vector of particles to build the octree from.

3.4.3.3 calc_force()

```
void Octree::calc_force (
    Particle & p)
```

Calculate the electrostatic force acting on `p` using the Barnes-Hut approximation and store the result in `p.acceleration`.

Compute the net electrostatic force on `p` via the Barnes-Hut approximation and update `p.acceleration` accordingly.

Parameters

<i>p</i>	The particle for which to compute the force. Modified in-place.
----------	---

The tree must already be populated (via [build\(\)](#) / [add_particle\(\)](#)). This function first calls [eval\(\)](#) on the root to (re-)compute aggregate charges and centres-of-charge, then performs the recursive traversal.

Parameters

<i>p</i>	The particle for which to compute the acceleration. Modified in-place.
----------	--

3.4.3.4 [get_root\(\)](#)

```
lazy_node< nt > & Octree::get_root ()
```

Return a mutable reference to the root node of the octree.

return the root node of the octree.

Note

Intended for testing and future scaling. In production code prefer the higher-level interface ([add_particle](#), [calc_force](#), [build](#)).

Returns

Reference to the root [lazy_node](#).

Note

This function is intended for future scaling purposes, the current implementation is intrusive for testing purposes and should be replaced with a encapsulated version.

3.4.3.5 [set_default_field\(\)](#)

```
void Octree::set_default_field (
    Octree def)
```

Replace the default field of the octree (reserved for future use).

Parameters

<i>def</i>	The default-field octree to apply.
------------	------------------------------------

3.4.3.6 set_theta()

```
void Octree::set_theta (
    float t) [inline]
```

Set the Barnes-Hut opening-angle parameter.

Parameters

<i>t</i>	Values closer to 0 give higher accuracy; typical default is 0.5.
----------	--

The documentation for this class was generated from the following files:

- [src/Octree.h](#)
- [src/Octree.cpp](#)

3.5 Particle Class Reference

Represents a single physical particle moving through the simulation domain.

```
#include <Particle.h>
```

Public Member Functions

- [Particle](#) ()
Default constructor. Creates a stationary, uncharged, dead particle at the origin.
- [Particle](#) (double x, double y, double z, double vx, double vy, double vz, std::string [type](#))
Construct a particle with position, velocity, and type. Mass and electric charge are inferred from `type`.
- [Particle](#) (double x, double y, double z, double vx, double vy, double vz, double m, int ec, std::string [type](#))
Construct a particle with position, velocity, explicit mass, charge, and type.
- void [move](#) (double dt)
Advance the particle by one time step dt . Updates velocity from acceleration, then position from velocity.

Public Attributes

- [vector3](#)< double > **position**
Current 3-D position of the particle (metres).
- [vector3](#)< double > **velocity**
Current 3-D velocity of the particle (metres per second).
- [vector3](#)< double > **acceleration**
Current 3-D acceleration of the particle (metres per second squared).
- double **mass**
Mass of the particle expressed in units of proton mass.
- int **electric_charge**
Electric charge of the particle in coulomb units (integer).
- std::string **type**
[Particle](#) species identifier (e.g. "proton" or "neutron").
- double **distance_in_shield**
Cumulative distance the particle has travelled inside a shielding volume (metres).
- bool **alive**
Whether the particle is still active in the simulation. Set to `false` once the particle is absorbed or leaves the domain.

3.5.1 Detailed Description

Represents a single physical particle moving through the simulation domain.

A [Particle](#) carries kinematic state (position, velocity, acceleration), physical properties (mass, electric charge, type), and bookkeeping information used by the simulation loop (distance travelled inside a shield, alive flag).

3.5.2 Constructor & Destructor Documentation

3.5.2.1 Particle() [1/3]

```
Particle::Particle ()
```

Default constructor. Creates a stationary, uncharged, dead particle at the origin.

Default constructor. Initialises all kinematic fields to zero and marks the particle as dead.

3.5.2.2 Particle() [2/3]

```
Particle::Particle (
    double x_,
    double y_,
    double z_,
    double vx_,
    double vy_,
    double vz_,
    std::string t)
```

Construct a particle with position, velocity, and type. Mass and electric charge are inferred from `type`.

Construct a particle from position, velocity, and type string. Mass and electric charge are inferred automatically from `t`.

Parameters

<code>x</code>	Initial x-coordinate (metres).
<code>y</code>	Initial y-coordinate (metres).
<code>z</code>	Initial z-coordinate (metres).
<code>vx</code>	Initial x-component of velocity (m/s).
<code>vy</code>	Initial y-component of velocity (m/s).
<code>vz</code>	Initial z-component of velocity (m/s).
<code>type</code>	Particle species. Accepted values: "proton", "neutron".

Exceptions

<code>std::runtime_error</code>	if <code>type</code> is not a recognised particle species.
---------------------------------	--

Parameters

$x \leftrightarrow$	Initial x-coordinate.
—	

$y_{\leftrightarrow}$ —	Initial y-coordinate.
$z_{\leftrightarrow}$ —	Initial z-coordinate.
$vx_{\leftrightarrow}$ —	Initial x-component of velocity.
$vy_{\leftrightarrow}$ —	Initial y-component of velocity.
$vz_{\leftrightarrow}$ —	Initial z-component of velocity.
t	Particle type string. Accepted values: "proton", "neutron".

Exceptions

<code>std::runtime_error</code>	if t is not a recognised particle type.
---------------------------------	---

3.5.2.3 Particle() [3/3]

```
Particle::Particle (
    double x_,
    double y_,
    double z_,
    double vx_,
    double vy_,
    double vz_,
    double m,
    int ec,
    std::string t)
```

Construct a particle with position, velocity, explicit mass, charge, and type.

Construct a particle with explicitly provided mass, charge, and type.

Parameters

x	Initial x-coordinate (metres).
y	Initial y-coordinate (metres).
z	Initial z-coordinate (metres).
vx	Initial x-component of velocity (m/s).
vy	Initial y-component of velocity (m/s).
vz	Initial z-component of velocity (m/s).
m	Mass in units of proton mass.
ec	Electric charge in coulomb units.
type	Particle species label.

$x_{\leftrightarrow}$ —	Initial x-coordinate.
$y_{\leftrightarrow}$ —	Initial y-coordinate.
$z_{\leftrightarrow}$ —	Initial z-coordinate.
$vx_{\leftrightarrow}$ —	Initial x-component of velocity.
$vy_{\leftrightarrow}$ —	Initial y-component of velocity.
$vz_{\leftrightarrow}$ —	Initial z-component of velocity.
m	Mass in units of proton mass.
ec	Electric charge in coulomb units.
t	Particle type label.

3.5.3 Member Function Documentation

3.5.3.1 move()

```
void Particle::move (
    double dt)
```

Advance the particle by one time step dt . Updates velocity from acceleration, then position from velocity.

Move the particle by updating its position and velocity based on its acceleration and the time step dt .

Parameters

dt	Time step duration (seconds). Must be strictly positive.
------	--

Exceptions

<code>std::runtime_error</code>	if dt is less than or equal to zero.
---------------------------------	--

Parameters

dt	The time step to move the particle. Must be positive and not equal to zero.
------	---

Exceptions

<code>std::runtime_error</code>	if dt is negative or zero.
---------------------------------	------------------------------

Note

this error can be ignored with a try catch if the final user wants to use a negative or zero time step.

This error is mainly here to prevent the user from accidentally using a negative or zero time step, which would lead to undefined behavior in the simulation.

The documentation for this class was generated from the following files:

- src/[Particle.h](#)
- src/[Particle.cpp](#)

3.6 ROI Class Reference

An axis-aligned bounding box used to score particles entering the region.

```
#include <ROI.h>
```

Public Member Functions

- [ROI](#) (double xmin, double xmax, double ymin, double ymax, double zmin, double zmax)
Construct a [ROI](#) from explicit axis-aligned bounds.
- bool [contains](#) (const [Particle](#) &p) const
Test whether a particle is currently inside the [ROI](#).
- void [score](#) ([Particle](#) &p)
Score a particle if it is alive and inside the [ROI](#). Increments `count` and marks the particle as dead to prevent double-counting.

Public Attributes

- [vector3](#)< double > **min_corner**
Minimum corner of the axis-aligned bounding box (metres).
- [vector3](#)< double > **max_corner**
Maximum corner of the axis-aligned bounding box (metres).
- int **count**
Cumulative count of particles scored inside this [ROI](#).

3.6.1 Detailed Description

An axis-aligned bounding box used to score particles entering the region.

The [ROI](#) counts each live [Particle](#) that enters its volume and marks that particle as no longer alive so it is not counted more than once.

3.6.2 Constructor & Destructor Documentation

3.6.2.1 ROI()

```
ROI::ROI (
    double xmin_,
    double xmax_,
    double ymin_,
    double ymax_,
    double zmin_,
    double zmax_)
```

Construct a [ROI](#) from explicit axis-aligned bounds.

Parameters

<i>xmin</i>	Minimum x-coordinate of the ROI (metres).
-------------	---

<i>xmax</i>	Maximum x-coordinate of the ROI (metres).
<i>ymin</i>	Minimum y-coordinate of the ROI (metres).
<i>ymax</i>	Maximum y-coordinate of the ROI (metres).
<i>zmin</i>	Minimum z-coordinate of the ROI (metres).
<i>zmax</i>	Maximum z-coordinate of the ROI (metres).
<i>xmin</i> ↔ —	Minimum x-coordinate (metres).
<i>xmax</i> ↔ —	Maximum x-coordinate (metres).
<i>ymin</i> ↔ —	Minimum y-coordinate (metres).
<i>ymax</i> ↔ —	Maximum y-coordinate (metres).
<i>zmin</i> ↔ —	Minimum z-coordinate (metres).
<i>zmax</i> ↔ —	Maximum z-coordinate (metres).

3.6.3 Member Function Documentation

3.6.3.1 contains()

```
bool ROI::contains (
    const Particle & p) const
```

Test whether a particle is currently inside the [ROI](#).

Return True if the particle is inside the [ROI](#), False otherwise.

Parameters

<i>p</i>	The particle to test.
----------	-----------------------

Returns

`true` if `p` is within the axis-aligned bounding box, `false` otherwise.

3.6.3.2 score()

```
void ROI::score (
    Particle & p)
```

Score a particle if it is alive and inside the [ROI](#). Increments `count` and marks the particle as dead to prevent double-counting.

If the particle is alive and inside the [ROI](#), increment the count and mark the particle as not alive anymore.

Parameters

<i>p</i>	The particle to evaluate. Modified in-place if scored.
----------	--

The documentation for this class was generated from the following files:

- [src/ROI.h](#)
- [src/ROI.cpp](#)

3.7 Shielding Class Reference

An axis-aligned rectangular volume of shielding material.

```
#include <Shielding.h>
```

Public Member Functions

- [Shielding](#) (double xmin, double xmax, double ymin, double ymax, double zmin, double zmax, [Material](#) m)
Construct a shielding volume from explicit axis-aligned bounds and a material.
- bool [contains](#) (const [Particle](#) &p) const
Test whether a particle is currently inside the shielding volume.

Public Attributes

- [vector3](#)< double > **min_corner**
Minimum corner of the axis-aligned bounding box (metres).
- [vector3](#)< double > **max_corner**
Maximum corner of the axis-aligned bounding box (metres).
- [Material](#) **material**
Physical material that fills this shielding volume.

3.7.1 Detailed Description

An axis-aligned rectangular volume of shielding material.

The [Shielding](#) class defines a box-shaped region of space that attenuates particles passing through it according to the mean free path of its material.

3.7.2 Constructor & Destructor Documentation

3.7.2.1 Shielding()

```
Shielding::Shielding (
    double xmin_,
    double xmax_,
    double ymin_,
    double ymax_,
    double zmin_,
    double zmax_,
    Material m)
```

Construct a shielding volume from explicit axis-aligned bounds and a material.

Parameters

<i>xmin</i>	Minimum x-coordinate of the shielding box (metres).
-------------	---

<i>xmax</i>	Maximum x-coordinate of the shielding box (metres).
<i>ymin</i>	Minimum y-coordinate of the shielding box (metres).
<i>ymax</i>	Maximum y-coordinate of the shielding box (metres).
<i>zmin</i>	Minimum z-coordinate of the shielding box (metres).
<i>zmax</i>	Maximum z-coordinate of the shielding box (metres).
<i>m</i>	Material that fills the shielding volume.

Exceptions

<i>std::invalid_argument</i>	if any min coordinate is greater than the corresponding max, or if min and max corners coincide (zero-volume box).
------------------------------	--

Parameters

<i>xmin</i> ↔ —	Minimum x-coordinate (metres).
<i>xmax</i> ↔ —	Maximum x-coordinate (metres).
<i>ymin</i> ↔ —	Minimum y-coordinate (metres).
<i>ymax</i> ↔ —	Maximum y-coordinate (metres).
<i>zmin</i> ↔ —	Minimum z-coordinate (metres).
<i>zmax</i> ↔ —	Maximum z-coordinate (metres).
<i>m</i>	Material that fills the volume.

Exceptions

<i>std::invalid_argument</i>	if any min coordinate exceeds the corresponding max, or if the min and max corners are identical (zero-volume box).
------------------------------	---

3.7.3 Member Function Documentation**3.7.3.1 contains()**

```
bool Shielding::contains (
    const Particle & p) const
```

Test whether a particle is currently inside the shielding volume.

Test whether a particle is currently inside the shielding volume. Uses normalised coordinates relative to the bounding box extents.

Parameters

p	The particle to test.
-----	-----------------------

Returns

`true` if `p` is within the shielding box, `false` otherwise.

Parameters

p	The particle to test.
-----	-----------------------

Returns

`true` if all normalised coordinates of `p` are in `[0, 1]`, `false` otherwise.

The documentation for this class was generated from the following files:

- [src/Shielding.h](#)
- [src/Shielding.cpp](#)

3.8 Simulation Class Reference

Top-level simulation driver.

```
#include <Simulation.h>
```

Public Member Functions

- [Simulation](#) (double `dt`, unsigned `steps`, [Source](#) `sr`, [Shielding](#) `s`, [ROI](#) `r`)
Construct a simulation with all required physical parameters.
- void [run](#) ()
Execute the simulation for `steps` time steps. Each step: emits new particles, moves all live particles, handles shielding absorption, scores the [ROI](#), and writes periodic CSV output.
- void [print](#) ()
Print a human-readable summary of the simulation parameters to `stdout`.

Public Attributes

- double `dt`
Time step duration in seconds.
- unsigned `steps`
Total number of time steps to simulate.
- [Source](#) `source`
The radioactive point source that emits particles each time step.
- `std::vector< Particle >` `particles`
List of particles currently active in the simulation.
- [Shielding](#) `shielding`
The shielding volume that attenuates particles passing through it.
- [ROI](#) `roi`
The region of interest used to score particles.

3.8.1 Detailed Description

Top-level simulation driver.

The [Simulation](#) class owns all physical objects (source, shielding, [ROI](#)) and the list of live particles. Calling [run\(\)](#) advances the simulation for the configured number of time steps, handling emission, movement, shielding absorption, [ROI](#) scoring, and periodic CSV output.

3.8.2 Constructor & Destructor Documentation

3.8.2.1 Simulation()

```
Simulation::Simulation (
    double dt_,
    unsigned steps_,
    Source src,
    Shielding s,
    ROI r)
```

Construct a simulation with all required physical parameters.

Construct a [Simulation](#) with the given physical parameters.

Parameters

dt	Time step duration (seconds).
steps	Number of time steps to run.
sr	Radioactive source configuration.
s	Shielding volume and material.
r	Region of interest for particle scoring.
dt_	Time step duration (seconds).
steps_	Total number of time steps to run.
src	Radioactive source configuration.
s	Shielding volume and material.
r	Region of interest for scoring particles.

3.8.3 Member Function Documentation

3.8.3.1 print()

```
void Simulation::print ()
```

Print a human-readable summary of the simulation parameters to `stdout`.

Print a human-readable summary of the simulation parameters to `stdout`. Includes time step, number of steps, total simulated time, source position and rate, shielding centre and material, and [ROI](#) centre.

3.8.3.2 run()

```
void Simulation::run ()
```

Execute the simulation for `steps` time steps. Each step: emits new particles, moves all live particles, handles shielding absorption, scores the [ROI](#), and writes periodic CSV output.

Execute the simulation for the configured number of time steps. new

The documentation for this class was generated from the following files:

- [src/Simulation.h](#)
- [src/Simulation.cpp](#)

3.9 Source Class Reference

Models a radioactive point source that emits particles every time step.

```
#include <Source.h>
```

Public Member Functions

- [Source](#) (double `x`, double `y`, double `z`, int `rate`, double `half_life`=1e9)
Construct a source at a given position with a given emission rate.
- `std::vector< Particle > emit () const`
Emit one batch of particles for the current time step. The number of particles returned equals `rate`. Each particle's direction and speed are sampled from normal distributions, and its type is chosen randomly (75 % proton, 25 % neutron).

Public Attributes

- double `x`
X-coordinate of the source position (metres).
- double `y`
Y-coordinate of the source position (metres).
- double `z`
Z-coordinate of the source position (metres).
- int `rate`
Number of particles emitted per time step (at $t = 0$).
- double `half_life`
Radioactive half-life in time steps. After `half_life` steps the effective emission rate halves. Defaults to 1e9 (essentially no decay).
- int `steps_elapsed` = 0
Internal step counter (mutable so [emit\(\)](#) can stay logically const).

3.9.1 Detailed Description

Models a radioactive point source that emits particles every time step.

The source is located at a fixed 3-D position and emits a given number of particles per time step. Each emitted particle has a randomised direction and speed drawn from statistical distributions, and its type (proton or neutron) is chosen randomly with a 3:1 proton-to-neutron ratio.

3.9.2 Constructor & Destructor Documentation

3.9.2.1 Source()

```
Source::Source (
    double x_,
    double y_,
    double z_,
    int rate_,
    double half_life_ = 1e9)
```

Construct a source at a given position with a given emission rate.

Construct a source at the specified position with the given emission rate.

Parameters

x	X-coordinate of the source (metres).
y	Y-coordinate of the source (metres).
z	Z-coordinate of the source (metres).
rate	Number of particles to emit per time step. Must be positive.
half_life	Decay half-life in time steps (default: effectively infinite).
<code>x_</code>	X-coordinate of the source (metres).
<code>y_</code>	Y-coordinate of the source (metres).
<code>z_</code>	Z-coordinate of the source (metres).
<code>rate_</code>	Number of particles to emit per time step.

3.9.3 Member Function Documentation

3.9.3.1 emit()

```
std::vector< Particle > Source::emit () const
```

Emit one batch of particles for the current time step. The number of particles returned equals `rate`. Each particle's direction and speed are sampled from normal distributions, and its type is chosen randomly (75 % proton, 25 % neutron).

Emit one batch of particles for the current time step. Direction angles (theta, phi) and speed norm are drawn from normal distributions to produce a realistic, slightly spread-out beam. [Particle](#) type is chosen randomly: 75 % proton, 25 % neutron.

Returns

A vector containing the newly emitted [Particle](#) objects.

A vector of `rate` newly created [Particle](#) objects, all originating from the source position.

The documentation for this class was generated from the following files:

- [src/Source.h](#)
- [src/Source.cpp](#)

3.10 `vector3< T >` Struct Template Reference

A generic 3-D vector whose component type is `T`.

```
#include <utils.h>
```

Public Member Functions

- `constexpr vector3 operator+ (const vector3 &n) const`
Element-wise addition.
- `constexpr void operator+= (const vector3 &n)`
In-place element-wise addition.
- `constexpr vector3 operator- (const vector3 &n) const`
Element-wise subtraction.
- `constexpr void operator-= (const vector3 &n)`
In-place element-wise subtraction.
- `constexpr vector3 operator* (const T &n) const`
*Scalar multiplication (vector * scalar).*
- `constexpr void operator*= (const T &n)`
In-place scalar multiplication.
- `constexpr T operator* (const vector3 &n) const`
Dot product.
- `constexpr void operator*= (const vector3 &n)`
In-place component-wise multiplication (replaces each component with the dot product).
- `constexpr vector3 operator/ (const T &n) const`
Scalar division.
- `constexpr vector3 operator/= (const T &n)`
In-place scalar division. Divides each component by `n` and stores the result in `*this`.
- `constexpr bool operator== (const vector3 &n) const`
Equality comparison.
- `double norm () const`
- `void normalize ()`
Normalize the vector.

Public Attributes

- `T x {}`
- `T y {}`
- `T z {}`

Friends

- constexpr **vector3 operator*** (const T &scalar, const **vector3** &v)
*Scalar multiplication (scalar * vector) — commutative companion to operator* (T).*
- std::ostream & **operator<<** (std::ostream &os, const **vector3** &vec)
Overload the << operator to print the vector in the format (x, y, z).

3.10.1 Detailed Description

```
template<typename T>
struct vector3< T >
```

A generic 3-D vector whose component type is T.

Template Parameters

<i>T</i>	Component type. Must be an integral or floating-point type (enforced via <code>static_assert</code>).
----------	--

Supports element-wise addition, subtraction, scalar multiplication/division, dot product, Euclidean norm, normalisation, equality comparison, and stream output.

3.10.2 Member Function Documentation

3.10.2.1 norm()

```
template<typename T>
double vector3< T >::norm () const [inline]
```

Returns

the eucliden norm of the vector as a double.

3.10.2.2 operator*() [1/2]

```
template<typename T>
vector3 vector3< T >::operator* (
    const T & n) const [inline], [constexpr]
```

Scalar multiplication (vector * scalar).

Parameters

<i>n</i>	Scalar multiplier.
----------	--------------------

Returns

Each component of `*this` multiplied by `n`.

3.10.2.3 operator*() [2/2]

```
template<typename T>
T vector3< T >::operator* (
    const vector3< T > & n) const [inline], [constexpr]
```

Dot product.

Parameters

<i>n</i>	Right-hand-side vector.
----------	-------------------------

Returns

Scalar dot product $x*n.x + y*n.y + z*n.z$.

3.10.2.4 operator*=() [1/2]

```
template<typename T>
void vector3< T >::operator*= (
    const T & n) [inline], [constexpr]
```

In-place scalar multiplication.

Parameters

<i>n</i>	Scalar multiplier.
----------	--------------------

3.10.2.5 operator*=() [2/2]

```
template<typename T>
void vector3< T >::operator*= (
    const vector3< T > & n) [inline], [constexpr]
```

In-place component-wise multiplication (replaces each component with the dot product).

Note

Due to the implementation delegating to the scalar dot-product `operator*(vector3)`, the resulting assignment of a scalar `T` to a `vector3` is unusual and may not compile for all types. Prefer explicit dot-product usage where needed.

Parameters

<i>n</i>	Right-hand-side vector.
----------	-------------------------

3.10.2.6 operator+()

```
template<typename T>
vector3 vector3< T >::operator+ (
    const vector3< T > & n) const [inline], [constexpr]
```

Element-wise addition.

Parameters

n	Right-hand-side vector.
-----	-------------------------

Returns

Component-wise sum of `*this` and `n`.

3.10.2.7 operator+=()

```
template<typename T>
void vector3< T >::operator+= (
    const vector3< T > & n) [inline], [constexpr]
```

In-place element-wise addition.

Parameters

n	Right-hand-side vector.
-----	-------------------------

3.10.2.8 operator-()

```
template<typename T>
vector3 vector3< T >::operator- (
    const vector3< T > & n) const [inline], [constexpr]
```

Element-wise subtraction.

Parameters

n	Right-hand-side vector.
-----	-------------------------

Returns

Component-wise difference `*this - n`.

3.10.2.9 operator-=()

```
template<typename T>
void vector3< T >::operator-= (
    const vector3< T > & n) [inline], [constexpr]
```

In-place element-wise subtraction.

Parameters

<i>n</i>	Right-hand-side vector.
----------	-------------------------

3.10.2.10 operator/()

```
template<typename T>
vector3 vector3< T >::operator/ (
    const T & n) const [inline], [constexpr]
```

Scalar division.

Parameters

<i>n</i>	Scalar divisor. Must not be zero.
----------	-----------------------------------

Returns

Each component of `*this` divided by `n`.

Exceptions

<code>std::runtime_error</code>	if <code>n</code> is zero.
---------------------------------	----------------------------

3.10.2.11 operator/=()

```
template<typename T>
vector3 vector3< T >::operator/= (
    const T & n) [inline], [constexpr]
```

In-place scalar division. Divides each component by `n` and stores the result in `*this`.

Note

The declared return type is `vector3` but the function body does not include an explicit `return` statement; treat this as an in-place mutation only.

Parameters

<i>n</i>	Scalar divisor.
----------	-----------------

3.10.2.12 operator==()

```
template<typename T>
bool vector3< T >::operator== (
    const vector3< T > & n) const [inline], [constexpr]
```

Equality comparison.

Parameters

<i>n</i>	Right-hand-side vector.
----------	-------------------------

Returns

`true` if all three components are equal.

3.10.3 Friends And Related Symbol Documentation

3.10.3.1 operator*

```
template<typename T>
vector3 operator* (
    const T & scalar,
    const vector3< T > & v) [friend]
```

Scalar multiplication (scalar * vector) — commutative companion to `operator*(T)`.

Parameters

<i>scalar</i>	Scalar multiplier.
<i>v</i>	Vector operand.

Returns

Each component of `v` multiplied by `scalar`.

The documentation for this struct was generated from the following file:

- [src/utils.h](#)

Chapter 4

File Documentation

4.1 Box.h

```
00001
00004
00005 #pragma once
00006
00007 #include "Particle.h"
00008 #include "utils.h"
00009
00010 class Box
00011 {
00012     private:
00013         vector3<double> min, max;
00014
00015     public:
00016
00017         Box(double x1, double x2, double y1, double y2, double z1, double z2)
00018         {
00019             if (x1 < x2)
00020             {
00021                 min.x = x1;
00022                 max.x = x2;
00023             }
00024             else
00025             {
00026                 min.x = x2;
00027                 max.x = x1;
00028             }
00029
00030             if (y1 < y2)
00031             {
00032                 min.y = y1;
00033                 max.y = y2;
00034             }
00035             else
00036             {
00037                 min.y = y2;
00038                 max.y = y1;
00039             }
00040
00041             if (z1 < z2)
00042             {
00043                 min.z = z1;
00044                 max.z = z2;
00045             }
00046             else
00047             {
00048                 min.z = z2;
00049                 max.z = z1;
00050             }
00051         }
00052
00053         Box(vector3<double> minVec, vector3<double> maxVec)
00054             : min(minVec), max(maxVec) {}
00055
00056         bool contains(const Particle& p) const
00057         {
00058             // implement the method that checks if a particle is inside the shielding
00059             // normalized coordinates
```

```

00060         vector3<double> n = (p.position - min) / (max - min).norm();
00061
00062         return n.x >= 0 && n.x <= 1 &&
00063                n.y >= 0 && n.y <= 1 &&
00064                n.z >= 0 && n.z <= 1;
00065     }
00066 };

```

4.2 src/GeometryOutput.cpp File Reference

Implementation of geometry output utilities: VTP mesh writing and global bounding-box particle culling.

```

#include "GeometryOutput.h"
#include <fstream>

```

Functions

- void [write_box_vtp](#) (double xmin, double xmax, double ymin, double ymax, double zmin, double zmax, const std::string &filename)
Write an axis-aligned box as a VTK PolyData (.vtp) file and update the global bounding box used by [is_in_bounding_box\(\)](#).
- void [is_in_bounding_box](#) (Particle &p)
Kill a particle that has left the global bounding box. Uses normalised coordinates; sets `p.alive = false` if the particle is outside `[0, 1]` in any dimension of the accumulated bounding box.

Variables

- double **bounding_xmin**
Module-level global bounding box updated by [write_box_vtp\(\)](#).
- double **bounding_xmax**
- double **bounding_ymin**
- double **bounding_ymax**
- double **bounding_zmin**
- double **bounding_zmax**

4.2.1 Detailed Description

Implementation of geometry output utilities: VTP mesh writing and global bounding-box particle culling.

4.2.2 Function Documentation

4.2.2.1 [is_in_bounding_box\(\)](#)

```

void is_in_bounding_box (
    Particle & p)

```

Kill a particle that has left the global bounding box. Uses normalised coordinates; sets `p.alive = false` if the particle is outside `[0, 1]` in any dimension of the accumulated bounding box.

Kill a particle if it has left the global bounding box.

Parameters

<code>p</code>	The particle to test and potentially deactivate.
----------------	--

4.2.2.2 write_box_vtp()

```
void write_box_vtp (
    double xmin,
    double xmax,
    double ymin,
    double ymax,
    double zmin,
    double zmax,
    const std::string & filename)
```

Write an axis-aligned box as a VTK PolyData (.vtp) file and update the global bounding box used by [is_in_bounding_box\(\)](#).

Write an axis-aligned box as a VTK PolyData (.vtp) file.

Parameters

<i>xmin</i>	Minimum x-coordinate of the box (metres).
<i>xmax</i>	Maximum x-coordinate of the box (metres).
<i>ymin</i>	Minimum y-coordinate of the box (metres).
<i>ymax</i>	Maximum y-coordinate of the box (metres).
<i>zmin</i>	Minimum z-coordinate of the box (metres).
<i>zmax</i>	Maximum z-coordinate of the box (metres).
<i>filename</i>	Path to the output .vtp file.

4.3 src/GeometryOutput.h File Reference

Declarations for geometry output utilities: writing bounding-box meshes in VTP format and enforcing the simulation domain boundary.

```
#include <string>
#include "Particle.h"
#include "Box.h"
```

Functions

- void [write_box_vtp](#) (double xmin, double xmax, double ymin, double ymax, double zmin, double zmax, const std::string &filename)
Write an axis-aligned box as a VTK PolyData (.vtp) file.
- void [is_in_bounding_box](#) ([Particle](#) &p)
Kill a particle if it has left the global bounding box.

4.3.1 Detailed Description

Declarations for geometry output utilities: writing bounding-box meshes in VTP format and enforcing the simulation domain boundary.

4.3.2 Function Documentation

4.3.2.1 `is_in_bounding_box()`

```
void is_in_bounding_box (
    Particle & p)
```

Kill a particle if it has left the global bounding box.

The global bounding box is the union of all boxes passed to `write_box_vtp()`. If `p` is outside this box its `alive` flag is set to `false`.

Parameters

<code>p</code>	The particle to test and potentially deactivate.
----------------	--

Kill a particle if it has left the global bounding box.

Parameters

<code>p</code>	The particle to test and potentially deactivate.
----------------	--

4.3.2.2 `write_box_vtp()`

```
void write_box_vtp (
    double xmin,
    double xmax,
    double ymin,
    double ymax,
    double zmin,
    double zmax,
    const std::string & filename)
```

Write an axis-aligned box as a VTK PolyData (.vtp) file.

The box is represented as a mesh of 8 vertices and 6 quad faces. Additionally, the function tracks the global bounding box of all boxes written so far (used by `is_in_bounding_box()`).

Parameters

<code>xmin</code>	Minimum x-coordinate of the box (metres).
<code>xmax</code>	Maximum x-coordinate of the box (metres).
<code>ymin</code>	Minimum y-coordinate of the box (metres).
<code>ymax</code>	Maximum y-coordinate of the box (metres).
<code>zmin</code>	Minimum z-coordinate of the box (metres).
<code>zmax</code>	Maximum z-coordinate of the box (metres).
<code>filename</code>	Path to the output .vtp file.

Write an axis-aligned box as a VTK PolyData (.vtp) file.

Parameters

<code>xmin</code>	Minimum x-coordinate of the box (metres).
-------------------	---

<i>xmax</i>	Maximum x-coordinate of the box (metres).
<i>ymin</i>	Minimum y-coordinate of the box (metres).
<i>ymax</i>	Maximum y-coordinate of the box (metres).
<i>zmin</i>	Minimum z-coordinate of the box (metres).
<i>zmax</i>	Maximum z-coordinate of the box (metres).
<i>filename</i>	Path to the output .vtp file.

4.4 GeometryOutput.h

[Go to the documentation of this file.](#)

```

00001
00004
00005 #pragma once
00006
00007 #include <string>
00008 #include "Particle.h"
00009 #include "Box.h"
00010
00024 void write_box_vtp(double xmin, double xmax,
00025                   double ymin, double ymax,
00026                   double zmin, double zmax,
00027                   const std::string& filename);
00028
00035 void is_in_bounding_box(Particle& p);

```

4.5 src/main.cpp File Reference

Entry point for the radioactive particle transport simulation.

```

#include "Simulation.h"
#include "Material.h"
#include "GeometryOutput.h"
#include <iostream>
#include <chrono>

```

Functions

- `int main ()`

4.5.1 Detailed Description

Entry point for the radioactive particle transport simulation.

Sets up the physical geometry (shielding, [ROI](#), source), writes initial VTP geometry files for visualisation, then constructs and runs the [Simulation](#). Execution time is measured and printed at the end.

4.6 src/Material.h File Reference

Declaration of the [Material](#) struct, which describes the physical properties of a shielding material relevant to particle attenuation.

```

#include <string>

```

Classes

- struct [Material](#)

Describes the physical properties of a shielding material.

4.6.1 Detailed Description

Declaration of the [Material](#) struct, which describes the physical properties of a shielding material relevant to particle attenuation.

4.7 Material.h

[Go to the documentation of this file.](#)

```
00001
00004
00005 #pragma once
00006
00007 #include <string>
00008
00013 struct Material {
00015     std::string name;
00016
00020     double mean_free_path;
00021
00025     Material(std::string n = "Vacuum", double mfp = 1e9)
00026         : name(n), mean_free_path(mfp) {}
00027 };
```

4.8 src/Octree.cpp File Reference

This file contains the implementation of the [Octree](#) class and its member functions.

```
#include "Octree.h"
#include <cmath>
#include <cstdint>
#include <cstdliblib>
```

4.8.1 Detailed Description

This file contains the implementation of the [Octree](#) class and its member functions.

Author

Maximilien Notz

4.9 src/Octree.h File Reference

This file contains the definition of the [Octree](#) class and [lazy_node](#).

```
#include "Particle.h"
#include "utils.h"
#include <cstdint>
#include <memory>
#include <type_traits>
```

Classes

- struct `lazy_node< T >`
A lazy node for the octree data structure.
- class `Octree`
A spatial partitioning octree data structure for implementing the Barnes-Hut algorithm.

Typedefs

- typedef double `nt`
The type of the coordinates of the octree. Must be a numeric type (int, float, double, etc.).

4.9.1 Detailed Description

This file contains the definition of the `Octree` class and `lazy_node`.

Author

Maximilien Notz

4.10 Octree.h

[Go to the documentation of this file.](#)

```
00001
00004
00005 #pragma once
00006
00007 #include "Particle.h"
00008 #include "utils.h"
00009
00010 #include <cstdint>
00011 #include <memory>
00012 #include <type_traits>
00013
00014
00026 template<typename T>
00027 struct lazy_node
00028 {
00029     static_assert(std::is_integral_v<T> || std::is_floating_point_v<T>);
00030
00031     struct
00032     {
00033         T length;
00034         vector3<T> center;
00035     } bounding_box;
00036
00037     vector3<T> center_of_charge;
00038     int total_charge; // The unit is coulombs
00039
00040     std::unique_ptr<lazy_node<T>> childs[8];
00041     std::shared_ptr<Particle> particle;
00042
00046     constexpr size_t child_index(const vector3<T>& p) const
00047     {
00048         if (p.x < bounding_box.center.x - bounding_box.length/2 ||
00049             p.y < bounding_box.center.y - bounding_box.length/2 ||
00050             p.z < bounding_box.center.z - bounding_box.length/2 ||
00051             p.x > bounding_box.center.x + bounding_box.length/2 ||
00052             p.y > bounding_box.center.y + bounding_box.length/2 ||
00053             p.z > bounding_box.center.z + bounding_box.length/2)
00054             throw std::runtime_error("lazy_node.child_index(p): Particle position must be within the
bounding box of the node.");
00055
00056         size_t child_index = 0;
00057
00058         // Determine the child index of the particle based on morton indices.
```

```

00059         if (p.x >= bounding_box.center.x)
00060             child_index = child_index | 0b001; // equivalent to child_index += 1;
00061         if (p.y >= bounding_box.center.y)
00062             child_index = child_index | 0b010; // equivalent to child_index += 2;
00063         if (p.z >= bounding_box.center.z)
00064             child_index = child_index | 0b100; // equivalent to child_index += 4;
00065         return child_index;
00066     }
00067 }
00068
00070 constexpr bool is_child_inst(const size_t& n) const { return childs[n] != nullptr; }
00071
00073 constexpr bool is_leaf() const
00074 {
00075     for (const auto& c : childs)
00076     {
00077         if (c != nullptr)
00078             return false;
00079     }
00080     return true;
00081 }
00082
00086 constexpr double Calc_BH_criteria(vector3<T> position) const { return bounding_box.length /
(center_of_charge - position).norm(); }
00087
00093 constexpr bool BH_criteria(const T& x, const T& y, const T& z, float theta) const { return
Calc_BH_criteria(vector3<T>{x,y,z}) < theta; }
00094
00098 constexpr bool BH_criteria(vector3<T> position, float theta) const { return
Calc_BH_criteria(position) < theta; }
00099
00101 inline void eval()
00102 {
00103     // recursion base case.
00104     if (is_leaf())
00105     {
00106         if (particle != nullptr)
00107         {
00108             center_of_charge = particle->position;
00109             total_charge = particle->electric_charge;
00110         }
00111         return;
00112     }
00113
00114     center_of_charge = vector3<T>{0, 0, 0};
00115     total_charge = 0;
00116
00117     for (std::unique_ptr<lazy_node<T>>& c: childs)
00118     {
00119         if (c != nullptr)
00120         {
00121             if (is_leaf() && c->particle != nullptr)
00122                 continue; // if the current node is a leaf and the child has a particle, we can
00123 skip the evaluation of the child since we will use the particle of the child to calculate the center
00124 of charge and total charge of the current node.;
00125
00126             c->eval();
00127             int combined = total_charge + c->total_charge;
00128             if (combined != 0)
00129                 center_of_charge = (total_charge*center_of_charge +
c->total_charge*c->center_of_charge)/combined;
00130             total_charge = combined;
00131         }
00132     }
00133 }
00134
00137 inline void eval(const size_t& n) { get(n).eval(); }
00138
00141 inline void instantiate_child(const size_t& n)
00142 {
00143     if (n > 7)
00144         throw std::runtime_error("lazy_node.instantiate_child(n): n must be smaller or equal to
7.");
00145
00146     childs[n] = std::make_unique<lazy_node<T>>();
00147     childs[n]->bounding_box.length = bounding_box.length / 2;
00148     childs[n]->bounding_box.center = bounding_box.center;
00149     childs[n]->bounding_box.center.x += (n & 1) ? bounding_box.length/4 : -bounding_box.length/4;
00150     childs[n]->bounding_box.center.y += (n & 2) ? bounding_box.length/4 : -bounding_box.length/4;
00151     childs[n]->bounding_box.center.z += (n & 4) ? bounding_box.length/4 : -bounding_box.length/4;
00152 }
00153
00156 inline const lazy_node& read(const size_t& n) const
00157 {
00158     if (n > 7)

```

```

00159         throw std::runtime_error("lazy_node.read(n): n must be smaller or equal to 7.");
00160     else if (childs[n] == nullptr)
00161         throw std::runtime_error("lazy_node.read(n): child is not instantiated.");
00162     return *childs[n];
00163 }
00164
00165 inline lazy_node& get(const size_t& n)
00166 {
00167     if (n > 7)
00168         throw std::runtime_error("lazy_node.get(n): n must be smaller or equal to 7.");
00169     else if (childs[n] == nullptr)
00170         instantiate_child(n); // create a dummy particle to instantiate the child if it is not
00171         already instantiated.
00172     return *childs[n];
00173 }
00174 }
00175 };
00176 };
00177
00178 typedef double nt;
00179
00180 class Octree
00181 {
00182 private:
00183     lazy_node<nt> root;
00184     float theta = 0.5f;
00185     static constexpr int max_depth = 50;
00186
00187     void add_particle(const Particle& p, lazy_node<nt>& node, int depth = 0);
00188
00189     void calc_force_recursive(const Particle& p, lazy_node<nt>& node,
00190                             vector3<double>& force);
00191
00192 public:
00193     Octree(vector3<nt> minVec, vector3<nt> maxVec);
00194
00195     void set_default_field(Octree def);
00196
00197     void add_particle(const Particle& p);
00198
00199     void calc_force(Particle& p);
00200
00201     void build(const std::vector<Particle>& particles);
00202
00203     void set_theta(float t) { theta = t; }
00204
00205     float get_theta() const { return theta; }
00206
00207     lazy_node<nt>& get_root();
00208 };

```

4.11 src/Output.cpp File Reference

Implementation of CSV output utilities for particle state persistence.

```

#include "Output.h"
#include <fstream>
#include <iostream>

```

Functions

- void [write_particles_csv](#) (const std::vector< [Particle](#) > &particles, const std::string &filename)
Write the state of a collection of particles to a CSV file.

4.11.1 Detailed Description

Implementation of CSV output utilities for particle state persistence.

4.11.2 Function Documentation

4.11.2.1 write_particles_csv()

```
void write_particles_csv (
    const std::vector< Particle > & particles,
    const std::string & filename)
```

Write the state of a collection of particles to a CSV file.

Creates or overwrites `filename` and writes a header row followed by one data row per particle with columns: `x`, `y`, `z`, `alive`, `type`.

Parameters

<i>particles</i>	The collection of particles to serialise.
<i>filename</i>	Path to the output CSV file.

Exceptions

<i>If</i>	the file cannot be opened an error message is printed to <code>stdout</code> and the exception is re-thrown.
-----------	--

4.12 src/Output.h File Reference

Declarations for CSV output utilities used to persist particle state.

```
#include <vector>
#include <string>
#include "Particle.h"
```

Functions

- void `write_particles_csv` (const std::vector< `Particle` > &particles, const std::string &filename)
Write the state of a collection of particles to a CSV file.

4.12.1 Detailed Description

Declarations for CSV output utilities used to persist particle state.

4.12.2 Function Documentation

4.12.2.1 write_particles_csv()

```
void write_particles_csv (
    const std::vector< Particle > & particles,
    const std::string & filename)
```

Write the state of a collection of particles to a CSV file.

The file will have a header row followed by one data row per particle. Columns: `x`, `y`, `z`, `alive`, `type`.

Parameters

<i>particles</i>	The collection of particles to serialise.
------------------	---

<i>filename</i>	Path to the output CSV file. The file is created or overwritten.
-----------------	--

Exceptions

<i>If</i>	the file cannot be opened, an error message is printed and an exception is re-thrown.
-----------	---

Creates or overwrites *filename* and writes a header row followed by one data row per particle with columns: *x*, *y*, *z*, *alive*, *type*.

Parameters

<i>particles</i>	The collection of particles to serialise.
<i>filename</i>	Path to the output CSV file.

Exceptions

<i>If</i>	the file cannot be opened an error message is printed to <i>stdout</i> and the exception is re-thrown.
-----------	--

4.13 Output.h

[Go to the documentation of this file.](#)

```
00001
00003
00004 #pragma once
00005
00006 #include <vector>
00007 #include <string>
00008 #include "Particle.h"
00009
00019 void write_particles_csv(const std::vector<Particle>& particles,
00020                        const std::string& filename);
```

4.14 src/Particle.cpp File Reference

Implementation of the [Particle](#) class methods.

```
#include "Particle.h"
```

4.14.1 Detailed Description

Implementation of the [Particle](#) class methods.

4.15 src/Particle.h File Reference

Declaration of the [Particle](#) class, representing a physical particle (e.g. proton or neutron) tracked throughout the simulation.

```
#include "utils.h"
#include <string>
```

Classes

- class [Particle](#)

Represents a single physical particle moving through the simulation domain.

4.15.1 Detailed Description

Declaration of the [Particle](#) class, representing a physical particle (e.g. proton or neutron) tracked throughout the simulation.

4.16 Particle.h

[Go to the documentation of this file.](#)

```
00001
00004
00005 #pragma once
00006
00007 #include "utils.h"
00008
00009 #include <string>
00010
00016 class Particle {
00017 public:
00019     vector3<double> position;
00020
00022     vector3<double> velocity;
00023
00025     vector3<double> acceleration;
00026
00028     double mass;
00029
00031     int electric_charge;
00032
00034     std::string type;
00035
00037     double distance_in_shield;
00038
00041     bool alive;
00042
00044     Particle();
00045
00056     Particle(double x, double y, double z,
00057             double vx, double vy, double vz,
00058             std::string type);
00059
00070     Particle(double x, double y, double z,
00071             double vx, double vy, double vz,
00072             double m,
00073             int ec,
00074             std::string type);
00075
00080     void move(double dt);
00081 };
```

4.17 src/ROI.cpp File Reference

Implementation of the [ROI](#) (Region Of Interest) class methods.

```
#include "ROI.h"
```

4.17.1 Detailed Description

Implementation of the [ROI](#) (Region Of Interest) class methods.

4.18 src/ROI.h File Reference

Declaration of the [ROI](#) (Region Of Interest) class, which defines an axis-aligned box used to score particles that pass through it.

```
#include "Particle.h"
```

Classes

- class [ROI](#)

An axis-aligned bounding box used to score particles entering the region.

4.18.1 Detailed Description

Declaration of the [ROI](#) (Region Of Interest) class, which defines an axis-aligned box used to score particles that pass through it.

4.19 ROI.h

[Go to the documentation of this file.](#)

```
00001
00004
00005 #pragma once
00006
00007 #include "Particle.h"
00008
00013 class ROI {
00014 public:
00016     vector3<double> min_corner;
00017
00019     vector3<double> max_corner;
00020
00022     int count;
00023
00031     ROI(double xmin, double xmax,
00032         double ymin, double ymax,
00033         double zmin, double zmax);
00034
00038     bool contains(const Particle& p) const;
00039
00043     void score(Particle& p);
00044 };
```

4.20 src/Shielding.cpp File Reference

Implementation of the [Shielding](#) class methods.

```
#include "Shielding.h"
```

4.20.1 Detailed Description

Implementation of the [Shielding](#) class methods.

4.21 src/Shielding.h File Reference

Declaration of the [Shielding](#) class, which models an axis-aligned rectangular shielding volume filled with a given material.

```
#include "Material.h"
#include "Particle.h"
#include "Box.h"
```

Classes

- class [Shielding](#)
An axis-aligned rectangular volume of shielding material.

4.21.1 Detailed Description

Declaration of the [Shielding](#) class, which models an axis-aligned rectangular shielding volume filled with a given material.

4.22 Shielding.h

[Go to the documentation of this file.](#)

```
00001
00004
00005 #pragma once
00006
00007 #include "Material.h"
00008 #include "Particle.h"
00009 #include "Box.h"
00010
00015 class Shielding
00016 {
00017 public:
00019     vector3<double> min_corner;
00020
00022     vector3<double> max_corner;
00023
00025     Material material;
00026
00037     Shielding(double xmin, double xmax,
00038               double ymin, double ymax,
00039               double zmin, double zmax,
00040               Material m);
00041
00045     bool contains(const Particle& p) const;
00046 };
```

4.23 src/Simulation.cpp File Reference

Implementation of the [Simulation](#) class methods.

```
#include "Simulation.h"
#include "utils.h"
#include "Output.h"
#include "Source.h"
#include "GeometryOutput.h"
#include <cmath>
#include <iostream>
#include <algorithm>
```

4.23.1 Detailed Description

Implementation of the [Simulation](#) class methods.

4.24 src/Simulation.h File Reference

Declaration of the [Simulation](#) class, which orchestrates the full radioactive particle transport simulation.

```
#include <vector>
#include "Particle.h"
#include "Shielding.h"
#include "ROI.h"
#include "Source.h"
#include "Octree.h"
```

Classes

- class [Simulation](#)
Top-level simulation driver.

4.24.1 Detailed Description

Declaration of the [Simulation](#) class, which orchestrates the full radioactive particle transport simulation.

4.25 Simulation.h

[Go to the documentation of this file.](#)

```
00001
00004
00005 #pragma once
00006
00007 #include <vector>
00008 #include "Particle.h"
00009 #include "Shielding.h"
00010 #include "ROI.h"
00011 #include "Source.h"
00012 #include "Octree.h"
00013
00020 class Simulation {
00021 public:
00023     double dt;
00024
00026     unsigned steps;
00027
00029     Source source;
00030
00032     std::vector<Particle> particles;
00033
00035     Shielding shielding;
00036
00038     ROI roi;
00039
00046     Simulation(double dt, unsigned steps, Source sr,
00047               Shielding s, ROI r);
00048
00052     void run();
00053
00055     void print();
00056 };
```

4.26 src/Source.cpp File Reference

Implementation of the [Source](#) class methods.

```
#include "Source.h"
#include "utils.h"
#include <cmath>
```

4.26.1 Detailed Description

Implementation of the [Source](#) class methods.

4.27 src/Source.h File Reference

Declaration of the [Source](#) class, which models a radioactive point source that emits particles isotropically at a configurable rate.

```
#include <vector>
#include <string>
#include "Particle.h"
```

Classes

- class [Source](#)

Models a radioactive point source that emits particles every time step.

4.27.1 Detailed Description

Declaration of the [Source](#) class, which models a radioactive point source that emits particles isotropically at a configurable rate.

4.28 Source.h

[Go to the documentation of this file.](#)

```
00001
00004
00005 #pragma once
00006
00007 #include <vector>
00008 #include <string>
00009 #include "Particle.h"
00010
00017 class Source {
00018 public:
00020     double x;
00021
00023     double y;
00024
00026     double z;
00027
00029     int rate;
00030
00034     double half_life;
00035
00037     mutable int steps_elapsed = 0;
00038
00045     Source(double x, double y, double z,
00046           int rate, double half_life = 1e9);
00047
00053     std::vector<Particle> emit() const;
00054 };
```

4.29 src/test.cpp File Reference

This file contains unit tests for the utility functions and data structures used in the simulation.

```
#include "dependencies/doctest.h"
#include "Particle.h"
#include "ROI.h"
#include "Octree.h"
#include <cmath>
```

Functions

- **TEST_SUITE** ("Vector3 struct")
- **TEST_SUITE** ("Particle")
- **TEST_SUITE** ("ROI")
- **TEST_SUITE** ("electrodynamic implementation via Barnes-Hut method")

4.29.1 Detailed Description

This file contains unit tests for the utility functions and data structures used in the simulation.

Note

doctest.h must be downloaded from the official doctest repository and placed in the dependencies folder for this file to compile and run correctly.

Author

Maximilien Notz

4.30 src/utils.h File Reference

Utility types and functions shared across the simulation.

```
#include <random>
#include <type_traits>
#include <stdexcept>
#include <cmath>
#include <fstream>
```

Classes

- struct [vector3< T >](#)
A generic 3-D vector whose component type is T.

Functions

- double **uniform01** ()
Return a random double in the range [0, 1) using a Mersenne Twister random number generator.
- template<typename T>
double **distance** (const **vector3**< T > &v0, const **vector3**< T > &v1)
Return the distance between v0 and v1.

4.30.1 Detailed Description

Utility types and functions shared across the simulation.

Provides:

- **vector3**<T>: a generic 3-D vector with arithmetic operators, norm, and normalisation.
- **uniform01**(): a convenience function returning a uniform random double in [0, 1).
- **distance**(): Euclidean distance between two **vector3** values.

4.30.2 Function Documentation

4.30.2.1 distance()

```
template<typename T>
double distance (
    const vector3< T > & v0,
    const vector3< T > & v1) [inline]
```

Return the distance between v0 and v1.

Template Parameters

<i>T</i>	The vector element data type.
----------	-------------------------------

Parameters

<i>v0</i>	the first vector.
<i>v1</i>	the second vector.

4.31 utils.h

[Go to the documentation of this file.](#)

```

00001
00008
00009 #pragma once
00010
00011 #include <random>
00012 #include <type_traits>
00013 #include <stdexcept>
00014 #include <cmath>
00015 #include <fstream>
00016
00024 template<typename T>
00025 struct vector3
00026 {
00027     T x{}, y{}, z{};
00028
00029     static_assert(std::is_integral_v<T> || std::is_floating_point_v<T>, "T must be a numeric type.");
00030
00034     constexpr vector3 operator+(const vector3 &n) const
00035     {
00036         vector3 sum;
00037         sum.x = x + n.x;
00038         sum.y = y + n.y;
00039         sum.z = z + n.z;
00040         return sum;
00041     }
00042
00045     constexpr void operator+=(const vector3 &n) { (*this) = (*this) + n; }
00046
00050     constexpr vector3 operator-(const vector3 &n) const
00051     {
00052         vector3 diff;
00053         diff.x = x - n.x;
00054         diff.y = y - n.y;
00055         diff.z = z - n.z;
00056         return diff;
00057     }
00058
00061     constexpr void operator-=(const vector3 &n) { (*this) = (*this) - n; }
00062
00066     constexpr vector3 operator*(const T &n) const
00067     {
00068         vector3 product;
00069         product.x = x * n;
00070         product.y = y * n;
00071         product.z = z * n;
00072         return product;
00073     }
00074
00075     // defines n * v, since the scalar product function above only defined for v * n.
00081     friend constexpr vector3 operator*(const T &scalar, const vector3 &v) { return v * scalar; }
00082
00085     constexpr void operator*=(const T &n) { (*this) = (*this) * n; }
00086
00090     constexpr T operator*(const vector3 &n) const { return x*n.x + y*n.y + z*n.z; }
00091
00097     constexpr void operator*=(const vector3 &n) { (*this) = (*this) * n; }
00098
00103     constexpr vector3 operator/(const T &n) const
00104     {
00105         if (n == 0)
00106             throw std::runtime_error("Vector Math error: Attempted to divide by Zero.");
00107
00108         vector3 ratio;
00109         ratio.x = x / n;
00110         ratio.y = y / n;
00111         ratio.z = z / n;
00112         return ratio;
00113     }
00114
00120     constexpr vector3 operator/=(const T &n) { (*this) = (*this) / n; }
00121
00125     constexpr bool operator==(const vector3 &n) const { return x == n.x && y == n.y && z == n.z; }
00126
00127
00129     friend std::ostream& operator<<(std::ostream& os, const vector3& vec)
00130     {
00131         os << "(" << vec.x << ", " << vec.y << ", " << vec.z << ")";
00132         return os;
00133     }
00134
00136     inline double norm() const { return std::sqrt((*this) * (*this)); }
00137

```

```
00139     inline void normalize()
00140     {
00141         T norm = static_cast<T>(this->norm());
00142         if (norm == 0)
00143             throw std::runtime_error("Vector Math error: Zero norm vectors cant be normalized.");
00144         *this = *this / norm;
00145     }
00146 };
00147 };
00148
00149 inline double uniform01()
00150 {
00151     static std::mt19937 gen(42);
00152     static std::uniform_real_distribution<double> dist(0.0, 1.0);
00153     return dist(gen);
00154 }
00155 }
00156
00157 template<typename T>
00158 inline double distance(const vector3<T>& v0, const vector3<T>& v1)
00159 {
00160     static_assert(std::is_integral_v<T> || std::is_floating_point_v<T>, "T must be a numeric type.");
00161     return std::sqrt((v1-v0) * (v1-v0));
00162 }
```

Index

- add_particle
 - Octree, [11](#)
- BH_criteria
 - lazy_node< T >, [6](#), [7](#)
- Box, [5](#)
- build
 - Octree, [11](#)
- Calc_BH_criteria
 - lazy_node< T >, [7](#)
- calc_force
 - Octree, [11](#)
- child_index
 - lazy_node< T >, [7](#)
- contains
 - ROI, [19](#)
 - Shielding, [21](#)
- distance
 - utils.h, [50](#)
- emit
 - Source, [25](#)
- eval
 - lazy_node< T >, [7](#)
- GeometryOutput.cpp
 - is_in_bounding_box, [34](#)
 - write_box_vtp, [34](#)
- GeometryOutput.h
 - is_in_bounding_box, [36](#)
 - write_box_vtp, [36](#)
- get
 - lazy_node< T >, [8](#)
- get_root
 - Octree, [12](#)
- instantiate_child
 - lazy_node< T >, [8](#)
- is_in_bounding_box
 - GeometryOutput.cpp, [34](#)
 - GeometryOutput.h, [36](#)
- lazy_node< T >, [5](#)
 - BH_criteria, [6](#), [7](#)
 - Calc_BH_criteria, [7](#)
 - child_index, [7](#)
 - eval, [7](#)
 - get, [8](#)
 - instantiate_child, [8](#)
 - read, [8](#)
- Material, [9](#)
 - Material, [9](#)
- move
 - Particle, [16](#)
- norm
 - vector3< T >, [27](#)
- Octree, [10](#)
 - add_particle, [11](#)
 - build, [11](#)
 - calc_force, [11](#)
 - get_root, [12](#)
 - Octree, [10](#)
 - set_default_field, [12](#)
 - set_theta, [12](#)
- operator+
 - vector3< T >, [28](#)
- operator+=
 - vector3< T >, [29](#)
- operator-
 - vector3< T >, [29](#)
- operator-=
 - vector3< T >, [29](#)
- operator/
 - vector3< T >, [30](#)
- operator/=
 - vector3< T >, [30](#)
- operator==
 - vector3< T >, [30](#)
- operator*
 - vector3< T >, [27](#), [31](#)
- operator*=
 - vector3< T >, [28](#)
- Output.cpp
 - write_particles_csv, [42](#)
- Output.h
 - write_particles_csv, [42](#)
- Particle, [13](#)
 - move, [16](#)
 - Particle, [14](#), [15](#)
- print
 - Simulation, [23](#)
- read
 - lazy_node< T >, [8](#)
- ROI, [17](#)
 - contains, [19](#)

- ROI, [18](#)
- score, [19](#)
- run
 - Simulation, [23](#)
- score
 - ROI, [19](#)
- set_default_field
 - Octree, [12](#)
- set_theta
 - Octree, [12](#)
- Shielding, [20](#)
 - contains, [21](#)
 - Shielding, [20](#)
- Simulation, [22](#)
 - print, [23](#)
 - run, [23](#)
 - Simulation, [23](#)
- Source, [24](#)
 - emit, [25](#)
 - Source, [25](#)
- src/Box.h, [33](#)
- src/GeometryOutput.cpp, [34](#)
- src/GeometryOutput.h, [35](#), [37](#)
- src/main.cpp, [37](#)
- src/Material.h, [37](#), [38](#)
- src/Octree.cpp, [38](#)
- src/Octree.h, [38](#), [39](#)
- src/Output.cpp, [41](#)
- src/Output.h, [42](#), [43](#)
- src/Particle.cpp, [43](#)
- src/Particle.h, [43](#), [44](#)
- src/ROI.cpp, [44](#)
- src/ROI.h, [45](#)
- src/Shielding.cpp, [45](#)
- src/Shielding.h, [46](#)
- src/Simulation.cpp, [46](#)
- src/Simulation.h, [47](#)
- src/Source.cpp, [48](#)
- src/Source.h, [48](#)
- src/test.cpp, [49](#)
- src/utils.h, [49](#), [51](#)
- utils.h
 - distance, [50](#)
- vector3< T >, [26](#)
 - norm, [27](#)
 - operator+, [28](#)
 - operator+=, [29](#)
 - operator-, [29](#)
 - operator-=, [29](#)
 - operator/, [30](#)
 - operator/=, [30](#)
 - operator==, [30](#)
 - operator*, [27](#), [31](#)
 - operator*==, [28](#)
- write_box_vtp
 - GeometryOutput.cpp, [34](#)
 - GeometryOutput.h, [36](#)
- write_particles_csv
 - Output.cpp, [42](#)
 - Output.h, [42](#)